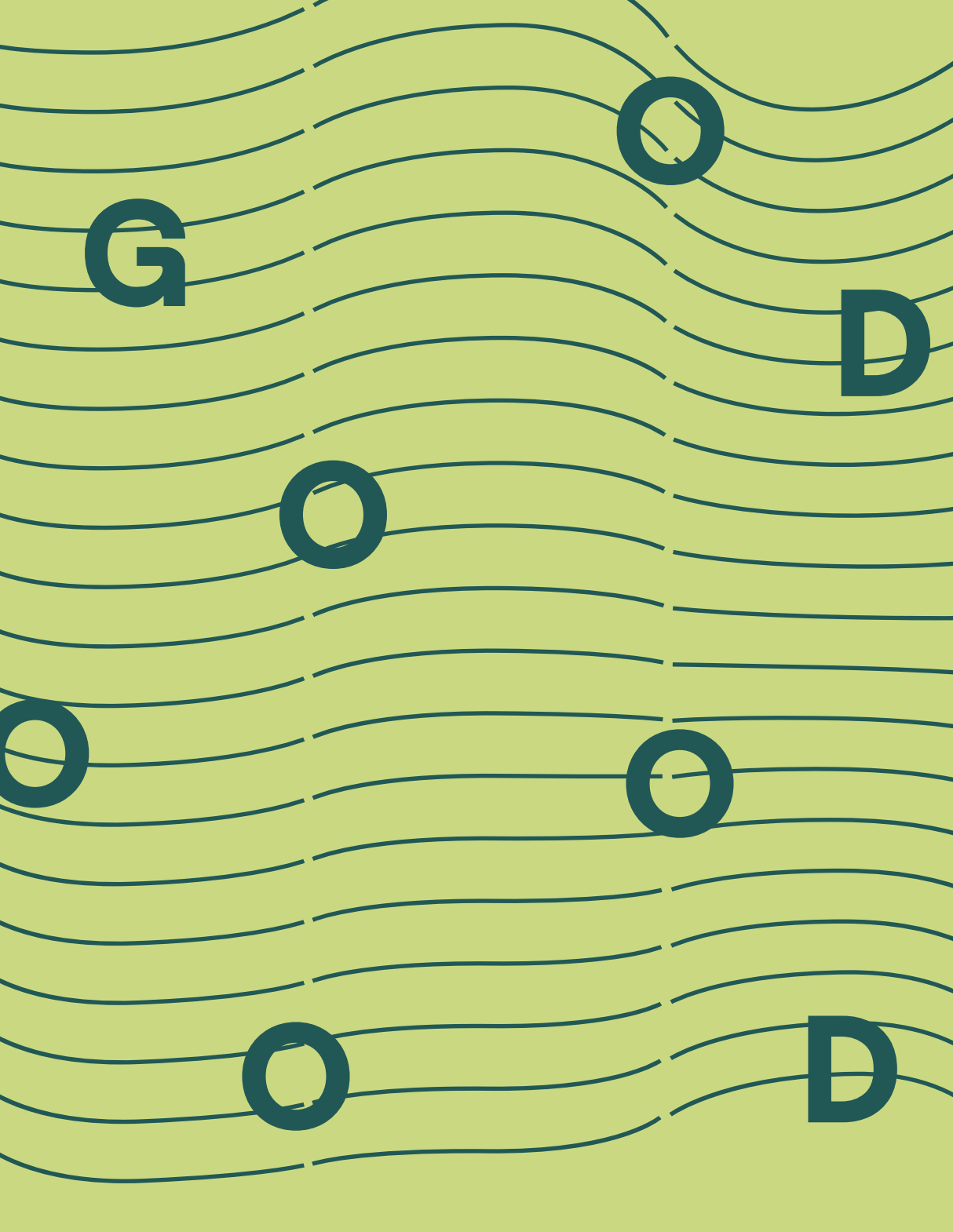




ROBIT c_Day5_hw1

2026406016 이동연

I. 포인터의 개념

- 메모리의 주소 (단위: 바이트)

```
int main(void)
{
    int i = 10;
}
```

주소	변수	변수값
100	i	10
101		
102		
103		
104		
105		
...		

int 자료형이 차지하는 메모리 공간: 4바이트

- char: 1, int: 4, float: 4, double: 8 ... (r.f Day 1 13p.)

II. 포인터 변수 선언

```
int main(void)
{
    int n = 5;
    int* pn = &n;
    :
}
```

III. 참조 연산자와 주소 연산자

변수 n의 주소를 의미: &n, & : 주소 연산자.

→ 즉 int* pn = &n 은 n의 주소를 포인터형 변수 pn에 저장한다는 의미.

```
printf("%d\n", *pn);
:
}
```

pn 주소에 저장된 값을 의미: *pn, *: 참조 연산자.

↓
n의 주소
↓
n = 5 이므로 5가 출력.

의미: int 형 변수의 주소를 담을 수 있는 포인터형 변수 pn을 선언한다.

구조: int* 포인터 변수이름

↳ char, float ... 등 다른 종류의 변수 주소를 담는 포인터형 변수를 선언하고 싶으면 char*, float* ... 처럼 선언할 수 있다.

IV. 특징.

• 함수 내부에서 함수 외부의 값을 가져오거나 수정 가능.

↳ 포인터를 배우기 이전 과제에서는 전역 변수 개념을 이용해 함수 내부에서 함수 외부의 값을 가져오거나 수정했으나 포인터를 이용하면, 메모리 효율성이 커지고, 규모가 큰 프로젝트들 진행할 때 더 유용하다.

• 1번 특징을 이용해 함수에서 두개 이상의 값을 반환할 수 있다.

↳ 기본에는 return 으로 하나만 가능.

↳ 미션까지도 전역변수를 이용해 두개 이상의 값을 반환하는 방법보다 효율적이다.

V. Call by Reference, 함수와 포인터

- 포인터 변수의 특징을 이용해 지역 변수의 한계를 해결할 수 있다.

o t.f C Day 4 실습 3

> f swap - CallByValue (int x, int y)

지역 변수로, 함수 내에서 x, y에 조작을 가해도 main 문에 영향을 미치지 못한다.

> f swap - CallByReference (int* px, int* py)

주소 받아서 주소에 담긴 내용물에 조작을 가하기 때문에 main 문에서 x, y가 정상적으로 swap 된다.

→ 주소는 일종의 고정된 전체 변수라고 이해해도 된 듯.

VI. 이중 포인터

- 포인터형 변수의 주소값을 저장하는 변수.

```

:
int n = 5;
int* np = &n;
int** p_np = &np
    
```

Name	n	np	p_np
Value	5	1000	2000
주소	1000	2000	3000 ...

VII. 배열과 포인터

배열의 크기를 전달해 함수가 배열의 최대 인덱스를 알 수 있도록 한다.

```

void f1(int* arr, int n)
{
    for(int i; i < n; i++)
    {
        : 배열 접근
    }
}
    
```

호출.

```

int main(void)
{
    int arr[10];
    f1(arr, 10);
    f1(&arr, 10);
    f1(&arr[0], 10);
}
    
```

- int arr[10]

4
0 1 2 3 4 5 6 7 8 9

↓ 배열 크기 (int int, ...)

시작 주소 : arr, &arr, &arr[0]

이차원 배열

→ $*(arr + i) + j$ 는 $arr[i][j]$ 와 같다. (t.f C Day 4 실습 4)

VIII 함수 포인터

- 함수의 주소들 저장하는 포인터

```
int add(int a, int b) {return a + b;}

int (*fp)(int, int); // 반환형 (*포인터 이름) (매개변수 타입들) 은 함수 포인터 선언
fp = add // add 함수의 주소 : add, &add
```

IX 구조체 포인터

```
struct Point
{
    int x;
    int y;
};

struct Point p = {10, 20};
struct Point* ptr = &p; // p의 주소 저장.

printf("%d\n", ptr->x); // 최상위 출력값: 10 출력
    "    , (*ptr).y); // 점으로 접근: 20 출력
```